

STLC Mapping: OrangeHRM Test Automation Project

Project: OrangeHRM Demo Automation **Author:** ABHISHEK K M **Status:** Baseline v1.0

Each phase below lists what happens *for this project specifically*, plus explicit entry criteria (what must be true to start) and exit criteria (what must be true to declare the phase done).

Phase 1 — Requirement Analysis

Activities for this project:

- Explore the OrangeHRM demo manually with the Admin/admin123 account and document actual behaviour per module (no SRS exists — the app is the spec).
- Identify testable requirements per module: **Login** — valid/invalid credentials, empty fields, "Forgot your password?" link, redirect to Dashboard on success. **Admin** — search/add/edit/delete system users, role and status filters, record-count validation. **PIM** — add employee (with/without login details), search employee, edit personal details, delete employee. **Leave** — apply leave, leave list filters, entitlement view. **Dashboard** — widget presence, quick-launch tiles, post-login landing assertions.
- Flag automation constraints: shared public environment, periodic data resets, no test API for seeding data (UI-only setup), no email verification possible.
- Classify requirements into Smoke / Regression candidates.

ENTRY CRITERIA

Demo application reachable and credentials verified. Project scope statement agreed (5 modules, Cypress + Playwright, JavaScript).

EXIT CRITERIA

Requirements catalogue per module reviewed and signed off by QA Lead. Each requirement tagged automatable / not-automatable with reason. Open questions logged with owners.

Phase 2 — Test Planning

Activities for this project:

- Produce the Test Plan (Artifact 3): scope, strategy, levels, environment, roles, risks, schedule, deliverables.
- Decide tooling and estimate effort per module; sequence modules (Login → Dashboard → Admin → PIM → Leave) so authentication utilities exist before dependent modules.
- Define defect-logging convention (GitHub Issues with severity/priority labels) and the flake-triage process.

ENTRY CRITERIA

Requirement Analysis exit criteria met.
Requirements catalogue available.

EXIT CRITERIA

Test Plan reviewed and approved. Effort estimate and schedule baselined. Risk register created with mitigations.

Phase 3 — Test Case Design

Activities for this project:

- Write manual test cases per module in Given/When/Then style, each with a unique ID (TC-LOG-001, TC-PIM-014, ...), priority, and Smoke/Regression tag — these become the single source of truth that both the Cypress and the Playwright suites implement.
- Design test data: static fixtures (valid credentials, invalid-credential matrix) and dynamic data rules (employee names suffixed with a timestamp to survive the shared environment).
- Perform review: peer review of test cases against the requirements catalogue; traceability matrix mapping requirement → test case ID.

ENTRY CRITERIA

Approved Test Plan. Requirements catalogue stable (no open blocking questions).

EXIT CRITERIA

100% of in-scope requirements traced to ≥1 test case. Test cases peer-reviewed; data design documented. Smoke subset identified (target: ≤10 min runtime).

Phase 4 — Test Environment Setup

Activities for this project:

- Local: Node LTS, Cypress and Playwright installed, browsers provisioned (npx playwright install), .env for credentials (never committed).

- Repo: folder structure per HLD, ESLint, package.json scripts (test:smoke, test:regression, per-module scripts).
- CI: GitHub Actions workflows (PR smoke, nightly regression), HTML report artifact upload.
- Environment validation ("smoke the environment"): a single login test must pass locally on all target browsers and in CI before any further authoring.

ENTRY CRITERIA

Test case design underway (at least Login cases ready). Repo access and CI runner availability confirmed.

EXIT CRITERIA

Environment-validation login test green locally (3 browsers) and in CI. Reporting artifact downloadable from a CI run. README setup instructions verified on a clean machine.

Phase 5 — Test Execution

Activities for this project:

- Implement and execute automated cases module by module; every case executed 3× locally before merge, then via CI.
- Log defects found in the AUT as GitHub Issues with repro steps, screenshots, and severity; distinguish AUT defects from script defects in triage.
- Track execution status per test case ID (Pass / Fail / Blocked / Skipped) against the traceability matrix; monitor the nightly regression trend.

ENTRY CRITERIA

Environment Setup exit criteria met.
Reviewed test cases available for the module under execution.

EXIT CRITERIA

All planned cases executed; results recorded against test case IDs. No open Critical/High script defects; AUT defects logged and classified. Flake rate < 2% over the last 10 CI regression runs.

Phase 6 — Test Closure

Activities for this project:

- Produce a Test Summary Report: coverage vs. plan, pass/fail statistics, defect summary, flake analysis, environment issues encountered.

- Retrospective: what slowed authoring (locator churn, demo resets), tooling comparison notes (Cypress vs. Playwright observations).
- Archive: tag the release, freeze the traceability matrix, ensure reports are stored as CI artifacts.
- Handover: README and docs sufficient for a new engineer to run and extend the suites.

ENTRY CRITERIA

Test Execution exit criteria met for all five modules.

EXIT CRITERIA

Test Summary Report published and reviewed. Retrospective actions logged. Repository tagged; documentation complete.

Summary table

STLC PHASE	KEY PROJECT ACTIVITY	HEADLINE EXIT CRITERION
Requirement Analysis	Explore app, build requirements catalogue	Catalogue signed off, automatability tagged
Test Planning	Test Plan, estimates, risk register	Plan approved, schedule baselined
Test Case Design	G/W/T cases + traceability matrix + data design	100% requirement coverage, peer-reviewed
Environment Setup	Local + CI + reporting pipeline	Login test green on 3 browsers + CI
Test Execution	Module-by-module automation + defect logging	All cases executed, flake < 2%
Test Closure	Summary report, retrospective, archive	Report published, repo tagged